CAPÍTULO

2

PROCESSO: UMA VISÃO GENÉRICA

CONCEITOS-CHAVE

arcabouço de processo 20
atividades guarda-chuva 20
avaliação de processo 29
CMMI 21
conjunto de tarefas 20
ISO 9001:2000 28
padrões de processo 24
PSP 29
tecnologia de processo 31
TSP 30

Em um livro fascinante, que fornece uma visão da economia do software e da engenharia de software, Howard Baetjer, Jr. [BAE98], faz os seguintes comentários sobre o processo de software:

> Desde que o software, como todo capital, é conhecimento incorporado, e como esse conhecimento está inicialmente disperso, tácito, latente e incompleto na sua totalidade, o desenvolvimento de software é um processo de aprendizado social. O processo é um diálogo no qual o conhecimento, que deve se transformar em software, é reunido e incorporado ao software. O processo fornece interação entre usuários e projetistas, entre usuários e ferramentas em desenvolvimento e entre projetistas e ferramentas em desenvolvimento [tecnologia]. É um processo iterativo no qual a própria ferramenta serve como meio de comunicação, com cada nova rodada de diálogo explicitando mais conhecimento útil do pessoal envolvido.

Efetivamente, a elaboração de software de computador é um processo iterativo de aprendizado, e o resultado, algo que Baetjer chamaria de "essência do software", é uma incorporação de conhecimentos coletados, destilados e organizados, à medida que o processo é conduzido.

Mas o que exatamente é um processo de software do ponto de vista técnico? Dentro do contexto deste livro, definimos *processo de software* como um arcabouço para as tarefas que são necessárias para construir softwares de alta qualidade. *Processo* é sinônimo de engenharia de software? A resposta é "sim" e "não". Um processo de software define a abordagem que é adotada quando o software é elaborado. Mas a engenharia de software também inclui tecnologias que constituem um processo — métodos técnicos e ferramentas automatizadas.

Mais importante, a engenharia de software é realizada por pessoas criativas, com amplos conhecimentos, que devem adaptar um processo de software maduro apropriado para os produtos que elas constroem e para as demandas do seu mercado.

PANORAMA

O que é? Quando você elabora um produto ou sistema é importante percorrer uma série de passos previsíveis — um roteiro que o ajuda a criar a tempo um resultado de alta qualidade. O roteiro que você segue é chamado de *processo de software*.

Quem faz? Os engenheiros de software e seus gerentes adaptam o processo a suas necessidades e depois o seguem. Além disso, o pessoal que solicitou o software tem um papel a desempenhar no processo de defini-lo, construí-lo e testá-lo.

Por que é importante? Porque fornece estabilidade, controle e organização para uma atividade que pode, se deixada sem controle, tornar-se bastante caótica. No entanto, uma abordagem moderna de engenharia de software precisa ser "ágil". Precisa exigir apenas aquelas atividades, controles e documentação adequados à equipe de projeto e ao produto que deve ser produzido.

Quais são os passos? Em nível de detalhe, o processo adotado depende do software que você está construindo. Um processo poderia ser apropriado à criação de softwares para o sistema de aviônica de uma aeronave, enquanto um processo inteiramente diferente seria indicado para a criação de um site.

Qual é o produto do trabalho? Do ponto de vista de um engenheiro de software, os produtos do trabalho são os programas, documentos e dados produzidos em conseqüência das atividades e tarefas definidas pelo processo.

Como tenho certeza de que fiz corretamente? Existem diversos mecanismos de avaliação do processo de software que permitem às organizações determinar a "maturidade" do seu processo de software. Todavia, a qualidade, pontualidade e viabilidade a longo prazo do produto que você constrói são os melhores indicadores da eficácia do processo usado.

2.1 Engenharia de Software — Uma Tecnologia em Camadas

Apesar de centenas de autores terem desenvolvido definições pessoais de *engenharia de software*, uma definição proposta por Fritz Bauer [NAU69] na conferência pioneira sobre o assunto ainda serve de base para a discussão:

> [Engenharia de software é] a criação e a utilização de sólidos princípios de engenharia a fim de obter softwares econômicos que sejam confiáveis e que trabalhem eficientemente em máquinas reais.

Quase todo leitor ficará tentado a ampliar essa definição. Ela diz pouco sobre os aspectos técnicos da qualidade do software; não inclui diretamente a necessidade de satisfação do cliente ou pontualidade; não faz menção à importância de medidas e unidades; não fala sobre a importância de um processo amadurecido. Ainda assim, a definição de Bauer nos dá uma linha básica. Quais são os "sólidos princípios de engenharia" que podem ser aplicados ao desenvolvimento de softwares de computador? Como construímos softwares "economicamente" e de modo que eles sejam "confiáveis"? O que é necessário para criar programas de computador que trabalhem "eficientemente" em não apenas uma, mas em várias "máquinas reais"? Essas são questões que continuam a desafiar os engenheiros de software.

> **"Mais que uma atividade ou um corpo de conhecimentos, a engenharia é um verbo, uma palavra de ação, uma forma de abordar um problema."**
>
> **Scott Whitmire**

? Como definir engenharia de software?

O IEEE [IEE93] desenvolveu uma definição mais abrangente ao estabelecer:

> Engenharia de software: (1) aplicação de uma abordagem sistemática, disciplinada e quantificável, para o desenvolvimento, operação e manutenção do software; isto é, a aplicação da engenharia ao software. (2) O estudo de abordagens como as de (1).

E no entanto, o que é "sistemático, disciplinado e quantificável" para uma equipe de software pode ser cansativo para outra. Precisamos de disciplina, mas precisamos também de adaptabilidade e agilidade.

PONTO CHAVE

A engenharia de software inclui um processo, métodos e ferramentas.

A engenharia de software é uma tecnologia em camadas. De acordo com a Figura 2.1, qualquer abordagem de engenharia (inclusive a engenharia de software) deve se apoiar num compromisso organizacional com a qualidade. Gestão de Qualidade Total, Seis Sigmas (Six Sigma) e filosofias análogas levam à cultura de um processo contínuo de aperfeiçoamento, e é essa cultura que, em última análise, leva ao desenvolvimento de abordagens cada vez mais efetivas para a engenharia de software. A base em que se apóia é o *foco na qualidade*.

O alicerce da engenharia de software é a camada de *processo*. O processo de engenharia de software é o adesivo que mantém unidas as camadas de tecnologia e permite o desenvolvimento racional e oportuno de softwares de computador. O processo define um arcabouço [PAU93] que deve ser estabelecido para a efetiva utilização da tecnologia de engenharia de software. Os processos de software formam a base para o controle gerencial de projetos de software e estabelecem o contexto no qual os métodos técnicos são aplicados, os produtos de trabalho (modelos, documentos, dados,

Figura 2.1

Engenharia de Software em camadas

Veja na Web

Cross Talk é um jornal que fornece informações pragmáticas sobre processo, métodos e ferramentas. Ele pode ser encontrado em **www.stsc.hill.af.mil.**

relatórios, formulários etc.) são produzidos, os marcos são estabelecidos, a qualidade é assegurada e as modificações são adequadamente geridas.

Os *métodos* de engenharia de software fornecem a técnica de "como fazer" para construir softwares. Eles abrangem um amplo conjunto de tarefas que incluem comunicação, análise de requisitos, modelagem de projeto, construção de programas, testes e manutenção. Os métodos de engenharia de software repousam num conjunto de princípios básicos que regem cada área da tecnologia e incluem atividades de modelagem e outras técnicas descritivas.

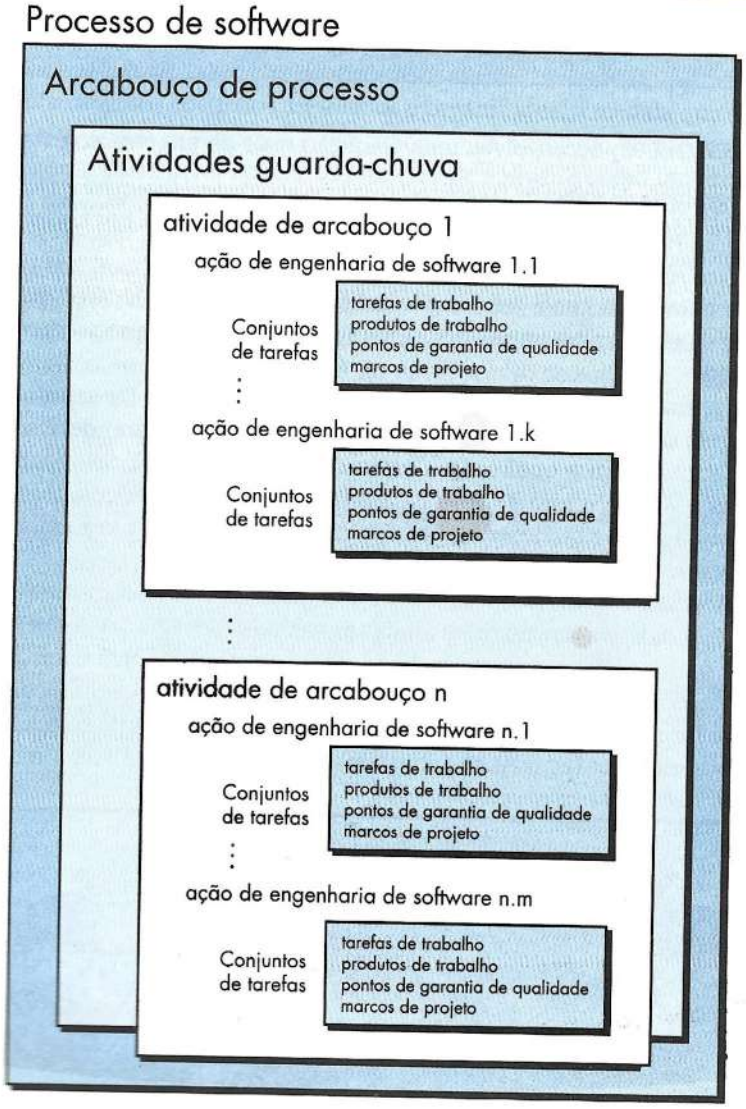
As *ferramentas* de engenharia de software fornecem apoio automatizado ou semi-automatizado para o processo e para os métodos. Quando ferramentas são integradas de modo que a informação criada por uma ferramenta possa ser usada por outra, um sistema de apoio ao desenvolvimento de software, chamado *engenharia de software apoiada por computador,* é estabelecido.

2.2 Um Arcabouço de Processo

Um *arcabouço de processo* estabelece o alicerce para um processo de software completo pela identificação de um pequeno número de *atividades de arcabouço* aplicáveis a todos os projetos de software, independentemente de seu tamanho ou complexidade. Além disso, o arcabouço de processo engloba um conjunto de *atividades guarda-chuva* que são aplicáveis durante todo o processo de software.

Figura 2.2

Um arcabouço de processo de software



Com referência à Figura 2.2, cada atividade de arcabouço é preenchida por um conjunto de *ações de engenharia de software* — uma coleção de tarefas relacionadas que produz um produto importante do trabalho de engenharia de software (por exemplo, o *projeto* é uma ação de engenharia de software). Cada ação é preenchida por *tarefas de trabalho* individuais que realizam alguma parte do trabalho determinado pela ação.

> **"Um processo define quem está fazendo *o quê, quando* e *como* para alcançar um certo objetivo."**
>
> **Ivar Jacobson, Grad Booch e James Rumbaugh**

O seguinte *arcabouço de processo genérico* (usado como base para a descrição dos modelos de processo nos capítulos subseqüentes) é aplicável à grande maioria dos projetos de software:

Quais são as cinco atividades genéricas do arcabouço de processo?

Comunicação. Essa atividade de arcabouço envolve alta comunicação e colaboração com o cliente (e outros interessados)[1] e abrange o levantamento de requisitos e outras atividades relacionadas.

Planejamento. Essa atividade estabelece um plano para o trabalho de engenharia de software que se segue. Descreve as tarefas técnicas a ser conduzidas, os riscos prováveis, os recursos que serão necessários, os produtos de trabalho a ser produzidos e um cronograma do trabalho.

Modelagem. Essa atividade inclui a criação de modelos que permitam ao desenvolvedor e ao cliente,entender melhor os requisitos do software e o projeto que vai satisfazer a esses requisitos.

Construção. Essa atividade combina geração de código (quer manual ou automática) e os testes necessários para revelar erros no código.

Implantação. O software (como entidade completa ou incremento parcialmente completo) é entregue ao cliente, que avalia o produto entregue e fornece *feedback* com base na avaliação.

Essas cinco atividades genéricas de arcabouço podem ser usadas durante o desenvolvimento de pequenos programas, durante a criação de grandes aplicações para a Internet e para a engenharia de grandes e complexos sistemas baseados em computador. Os detalhes do processo de software serão muito diferentes em cada caso, mas as atividades de arcabouço permanecem as mesmas.

> **"Einstein argumentou que deve haver uma explicação simplificada da natureza, porque Deus não é inconstante ou arbitrário. Nenhuma dessas fés conforta o engenheiro de software. Muito da complexidade que ele deve controlar é complexidade arbitrária."**
>
> **Fred Brooks**

PONTO CHAVE

Diferentes projetos necessitam de diferentes conjuntos de tarefas. a equipe de software escolhe o conjunto de tarefas com base no problema e características do projeto.

Usando um exemplo derivado do arcabouço de processo genérico, a *atividade de modelagem* é composta de duas ações de engenharia de software — *análise* e *projeto*. A análise[2] engloba um conjunto de tarefas de trabalho (por exemplo, levantamento, elaboração, negociação, especificação e validação de requisitos) que conduzem à criação de modelos de análise (e/ou especificação de requisitos). O projeto engloba tarefas de trabalho (projeto de dados, projeto arquitetural, projeto de interface e projeto de componentes) que criam um modelo (e/ou uma especificação de projeto).[3]

Com referência novamente à Figura 2.2, cada ação de engenharia de software é representada por um número diferente de *conjuntos de tarefas* — cada um como uma coleção de tarefas de trabalho de engenharia de software, produtos de trabalho relacionados, pontos de garantia de qualidade e

1 Um "interessado" é alguém que tem interesse no resultado bem-sucedido do projeto — gerentes de negócio, usuários finais, engenheiros de software, pessoal de apoio etc. Rob Thomsett brinca que "um interessado é uma pessoa empunhando uma estaca grande e afiada (...) se você não prestar atenção nos seus interessados, você sabe onde a estaca vai acabar ficando".

* N.R.T. Essa brincadeira é porque o termo usado para "interessado" em inglês, *stakeholder*, tem o sentido literal de "porta-estaca".

2 A análise será discutida em profundidade nos Capítulos 7 e 8.

3 Deve-se notar que a "modelagem" deve ser interpretada de modo diferente quando a manutenção de um software existente é realizada. Em alguns casos, a modelagem de análise e do projeto de fato ocorre, mas em outras situações de manutenção, a modelagem pode ser usada para ajudar a entender o software legado bem como para representar adições ou modificações nele.

marcos de projeto. O conjunto de tarefas escolhido é o que melhor acomoda as necessidades do projeto e as características da equipe. Isso implica que uma ação de engenharia de software (por exemplo, o projeto) pode ser adaptada às necessidades específicas do projeto de software e às características da equipe do projeto.

INFO

Conjunto de Tarefas

Um conjunto de tarefas define o trabalho real a ser feito para atingir os objetivos de uma ação de engenharia de software. Por exemplo, "levantamento de requisitos" é uma importante ação de engenharia de software que ocorre durante a atividade **comunicação**. O objetivo do levantamento de requisitos é entender o que vários interessados desejam do software que está sendo construído.

Para um projeto pequeno e relativamente simples, o conjunto de tarefas para o levantamento de requisitos pode ter o seguinte aspecto:

1. Faça uma lista de interessados no projeto.
2. Convide todos os interessados para uma reunião informal.
3. Peça a cada interessado para fazer uma lista das características e funções desejadas.
4. Discuta os requisitos e construa uma lista final.
5. Priorize os requisitos.
6. Observe áreas de incerteza.

Para um projeto de software maior e mais complexo, um conjunto de tarefas diferente seria necessário. Ele pode incluir as seguintes tarefas de trabalho:

1. Faça uma lista de interessados no projeto.
2. Entreviste cada interessado separadamente para determinar seus desejos e necessidades gerais.
3. Construa uma lista preliminar de funções e características com base nas informações fornecidas pelos interessados.
4. Programe uma série de reuniões facilitadas de levantamento de requisitos.
5. Realize as reuniões.
6. Produza cenários informais de usuário como parte de cada reunião.
7. Refine os cenários de usuário com base no feedback dos interessados.
8. Construa uma lista revisada de requisitos dos interessados.
9. Use técnicas de implantação de função de qualidade para priorizar os requisitos.
10. Empacote os requisitos de modo que eles possam ser entregues incrementalmente.
11. Anote as restrições e limitações que serão colocadas no sistema.
12. Discuta métodos para validação do sistema.

Ambos os conjuntos de tarefas realizam o levantamento de requisitos, mas eles são muito diferentes em profundidade e formalidade. A equipe de software escolhe o conjunto de tarefas que lhe permitir alcançar o objetivo de cada atividade de processo e a ação de engenharia de software e, ainda, manter a qualidade e a agilidade.

O arcabouço descrito na visão genérica de engenharia de software é complementado por várias atividades *guarda-chuva*. Atividades típicas dessa categoria incluem:

PONTO CHAVE

Atividades guarda-chuva ocorrem ao longo do processo de software e focalizam principalmente a gestão, o monitoramento e o controle do projeto.

Acompanhamento e controle de projeto de software — permite à equipe de software avaliar o progresso com base no plano de projeto e tomar a ação necessária para manter o cronograma.

Gestão de risco — avalia os riscos que podem afetar o resultado do projeto ou a qualidade do produto.

Garantia de qualidade de software — define e conduz as atividades necessárias para garantir a qualidade do software.

Revisões técnicas formais — avaliam os produtos de trabalho da engenharia de software, num esforço para descobrir e remover erros antes que eles sejam propagados para a próxima ação ou atividade.

Medição — define e reúne medidas de processo, projeto e produto que ajudam a equipe a entregar um software que satisfaça às necessidades do usuário; pode ser usada conjugada com todas as outras atividades de arcabouço e guarda-chuva.

Gestão de configuração de software — gerencia os efeitos das modificações ao longo de todo o processo de software.

Gestão de reusabilidade — define critérios para a reutilização dos produtos de trabalho (inclusive componentes de software) e estabelece mecanismos para obter componentes reusáveis.

Preparação e produção do produto do trabalho — abrange as atividades necessárias para criar produtos do trabalho como modelos, documentos, registros, formulários e listas.

As atividades guarda-chuva são aplicadas ao longo de todo o processo de software e serão discutidas em detalhes mais adiante neste livro.

Todos os modelos de processo podem ser caracterizados dentro do arcabouço de processo mostrado na Figura 2.2. A aplicação inteligente de qualquer modelo de processo de software deve reconhecer que a adaptação (ao problema, ao projeto, à equipe e à cultura organizacional) é essencial para o sucesso. Mas os modelos de processo diferem fundamentalmente:

PONTO CHAVE

A adaptação de processos de software é essencial para o sucesso do projeto.

- No fluxo geral de atividades e tarefas e interdependências entre atividades e tarefas.
- No grau em que as tarefas de trabalho são definidas dentro de cada atividade de arcabouço.
- No grau em que os produtos do trabalho são identificados e solicitados.
- Na maneira como as atividades de garantia de qualidade são aplicadas.
- No modo como o monitoramento de projeto e as atividades de controle são aplicados.
- No grau geral de detalhes e rigor em que o processo é descrito.
- No grau em que clientes e outros interessados estão envolvidos no projeto.
- No nível de autonomia da equipe de projeto de software.
- No grau em que a organização da equipe e os papéis são prescritos.

? Como os modelos de processo diferem uns dos outros?

> "Sinto que uma receita é apenas um tema que um cozinheiro inteligente pode usar cada vez com uma variante."
>
> **Madame Benoit**

Modelos de processo que enfatizam a definição, identificação e aplicação detalhada de atividades e tarefas de processo têm sido aplicados na comunidade de engenharia de software durante os últimos trinta anos. Quando esses *modelos prescritivos de processo* são aplicados, o objetivo é melhorar a qualidade do sistema para tornar os projetos mais gerenciáveis, as datas de entrega e os custos mais previsíveis e para guiar equipes de engenheiros de software à medida que eles realizam o trabalho necessário para construir um sistema. Infelizmente, tem havido épocas em que esses objetivos não têm sido alcançados. Se os modelos prescritivos forem aplicados dogmaticamente e sem adaptação, eles podem aumentar o nível de burocracia associado à construção de sistemas baseados em computador e, inadvertidamente, criar dificuldades para desenvolvedores e clientes.

? O que caracteriza um processo "ágil"?

Modelos de processo que enfatizam a agilidade do projeto e seguem uma série de princípios[4] que levam a uma abordagem mais informal (mas, segundo seus proponentes, não menos efetiva) do processo de software foram propostos no últimos anos. Esses *modelos de processo ágil* enfatizam a manobrabilidade e a adaptabilidade. Eles são adequados a muitos tipos de projeto e são particularmente úteis quando aplicações Web passam por engenharia.

Que filosofia de processo de software é melhor? Essa questão tem provocado um debate emocional entre engenheiros de software e será discutida no Capítulo 4. Por hora, é importante notar que essas duas filosofias de processo têm um objetivo comum — criar softwares de alta qualidade que satisfaçam às necessidades do cliente —, mas abordagens diferentes.

2.7 Tecnologia de Processo

Os modelos de processo genéricos discutidos nas seções precedentes devem ser adaptados para o uso de uma equipe de projeto de software. Para conseguir isso, foram desenvolvidas *ferramentas de tecnologia de processo* a fim de ajudar as organizações a analisar seu processo atual, organizar tarefas de trabalho, controlar e monitorar o progresso e gerenciar a qualidade técnica [NEG99].

As ferramentas de tecnologia de processo permitem a uma organização de software construir um modelo automatizado do arcabouço comum de processo, dos conjuntos de tarefas e das atividades guarda-chuva discutidas na Seção 2.2. O modelo, normalmente representado por uma rede, pode então ser analisado para determinar o fluxo típico de trabalho e examinar estruturas de processo alternativas que possam reduzir o tempo de desenvolvimento ou o custo.

Uma vez criado um processo aceitável, outras ferramentas de tecnologia de processo podem ser usadas para atribuir, monitorar e até mesmo controlar todas as tarefas de engenharia de software definidas como parte do modelo de processo. Cada membro de uma equipe de software pode usar

tais ferramentas para desenvolver um *checklist* das tarefas de trabalho a ser desenvolvidas, dos produtos de trabalho a ser produzidos e das atividades de garantia de qualidade a ser conduzidas. A ferramenta de tecnologia de processo também pode ser usada para coordenar o uso de outras ferramentas de engenharia de software apoiadas por computador que sejam adequadas a uma tarefa de trabalho específica.

FERRAMENTAS DE SOFTWARE

Ferramentas de Modelagem de Processo

Objetivo: Se uma organização trabalha para aperfeiçoar um processo de negócio (ou software), ela deve primeiramente entendê-lo. As ferramentas de modelagem de processo (também chamadas de *ferramentas de tecnologia de processo ou gestão de processo*) são usadas para representar os elementos-chave de um processo de modo que ele possa ser melhor entendido. Tais ferramentas podem também fornecer links para descrições do processo que ajudem os que estão envolvidos no processo a entender as ações e tarefas de trabalho necessárias para realizá-lo. Ferramentas de modelagem de processo fornecem links para outras ferramentas que fornecem apoio a atividades definidas do processo.

Mecânica: As ferramentas dessa categoria permitem a uma equipe definir os elementos de um modelo de processo específico (ações, tarefas, produtos de trabalho, pontos de garantia de qualidade), fornecer diretrizes detalhadas sobre o conteúdo ou a descrição de cada elemento do processo e, depois, gerir o projeto à medida que ele é conduzido. Em alguns casos, as ferramentas de tecnologia de processo incorporam tarefas-padrão de gestão de projeto tais como orçar, montar cronogramas, monitorar e controlar.

Ferramentas Representativas: [11]

As ferramentas *Igrafx Process Tools*, distribuídas pela Corel Corporation (www.igrafx.com/products/process), são um conjunto de ferramentas que permitem a uma equipe mapear, medir e modelar o processo de software.

O *Objexis Team Portal*, desenvolvido pela Objexis Corporation (www.objexis.com), fornece definição e controle para todo o fluxo de trabalho do processo.

2.8 PRODUTO E PROCESSO

Se o processo for fraco, o produto final sofrerá inevitavelmente, mas uma ênfase exagerada no processo também é perigosa. Em um ensaio resumido, Margaret Davis [DAV95] comenta a dualidade do produto e do processo:

> A cada dez anos, avançando ou recuando cinco, a comunidade de software redefine "o problema" mudando o foco de aspectos do produto para aspectos do processo. Assim, adotamos linguagens de programação estruturada (produto), seguidas de métodos de análise estruturada (processo) e, depois, do encapsulamento dos dados (produto), da ênfase atual no Modelo de Maturidade da Capacidade de Desenvolvimento de Software, do SEI (Software Engineering Institute) (processo) [seguido por métodos orientados a objetos e desenvolvimento ágil de softwares].

Enquanto um pêndulo tende a ficar naturalmente em repouso no ponto intermediário entre dois extremos, o foco da comunidade de software desloca-se constantemente, pois uma nova força é aplicada quando a última oscilação falha. Essas oscilações são prejudiciais em si e por si, porque confundem o profissional de software médio modificando radicalmente o significado de realizar o trabalho, ou de realizá-lo bem. As oscilações também não resolvem "o problema", pois estão fadadas a falhar, enquanto produto e processo forem tratados como uma dicotomia em vez de uma dualidade.

Há precedência na comunidade científica para sugerir noções de dualidade quando contradições nas observações não podem ser totalmente explicadas por uma ou outra teoria competitiva. A natureza dual da luz, que parece ser simultaneamente partícula e onda, tem sido aceita desde 1920,

11 As ferramentas indicadas aqui não representam uma recomendação, mas sim uma exemplificação de ferramentas nessa categoria. Na maioria dos casos, os nomes das ferramentas são marcas registradas de seus respectivos desenvolvedores.

quando Louis de Broglie a propôs. Acredito que as observações que podemos fazer das ferramentas de software e do seu desenvolvimento demonstram uma dualidade fundamental entre produto e processo. Não se pode deduzir ou entender toda a ferramenta, seu contexto, uso, significado e valor se a virmos somente como um processo ou somente como um produto...

Toda atividade humana pode ser um processo, mas cada um de nós obtém um senso de auto-valor daquelas atividades que resultam numa representação ou instância, que pode ser usada ou apreciada por mais de uma pessoa, utilizada repetidamente ou empregada em algum outro contexto não considerado. Isto é, obtemos um sentimento de satisfação pelo reuso de nossos produtos, por nós mesmos ou pelos outros.

Assim, enquanto a rápida assimilação de metas de reutilização no desenvolvimento de software aumenta potencialmente a satisfação dos profissionais de software com seu trabalho, ela aumenta também a urgência da aceitação da dualidade entre produto e processo. Pensando em um artefato reutilizável somente como um produto ou somente como um processo, ou obscurece o contexto e os modos de usá-lo, ou obscurece o fato de que cada uso resulta num produto que, por sua vez, será usado como entrada para alguma outra atividade de desenvolvimento de software. Adotar uma visão ao invés da outra reduz substancialmente as oportunidades de reutilização e, conseqüentemente, as oportunidades de satisfação crescente com o trabalho.

> **"Sem dúvida, o sistema ideal, se fosse passível de obtenção, seria um código ao mesmo tempo tão flexível e tão pequeno, que seria capaz de fornecer previamente, para cada situação concebível, uma regra própria e adequada. Mas a vida é complexa demais para trazer o alcance dessa idéia para o âmbito do poder humano."**
>
> **Benjamin Cardozo**

As pessoas obtêm tanta (ou mais) satisfação de um processo criativo quanto de um produto final. Um artista aprecia tanto as pinceladas quanto o resultado emoldurado. Um escritor aprecia tanto a busca de uma metáfora adequada quanto o livro pronto. Um profissional de software criativo também deveria obter tanta satisfação do processo quanto do produto final.

O trabalho do pessoal de software vai se modificar nos próximos anos. A dualidade entre produto e processo é um elemento importante para manter pessoas criativas engajadas enquanto a transição da programação para a engenharia de software é concluída.